

PRACTICAL 7 & 8

Practical 7

```
#include <stdio.h>
#include <stdlib.h>
int global=100;
static int s=50;

int main()
{
int stack=10;
int *heap=(int*)malloc(sizeof(int));

printf("Code Segment: %p\n",main);
printf("Global Segment : %p n",&global);
printf("Static Segment : %pn",&s);
printf("Heap Segment: %pn",heap);
printf("Stack Segment : %pn",&stack);
free(heap);
return 0;
}
```

Output: -

```
shashank@LAPTOP-24KPR0R8:~$ gcc Address.c -o Address
shashank@LAPTOP-24KPR0R8:~$ ./Address
Code Segment: 0x56933d2d71a9
Global Segment : 0x56933d2da010 nStatic Segment : 0x56933d2da014nHeap Segmen
t: 0x569375b1a010nStack Segment : 0x7ffe9da13ddcnshashank@LAPTOP-24KPR0R8:~$
```

Practical 8

```
GNU nano 8.7.1 Valgrind
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int*)malloc(5 * sizeof(int));
    b = (int*)calloc(5, sizeof(int));
    a = (int*)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

b)

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int x = 100;

    if (fork() == 0)
    {
        printf("Child Before = %d\n", x);
        x = 500;
        printf("Child After = %d\n", x);
    }
    else
    {
        sleep(2);
        printf("Parent = %d\n", x);
    }

    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPR0R8:~$ gcc Copywrite.c -o Copywrite
shashank@LAPTOP-24KPR0R8:~$ ./Copywrite
Child Before = 100
Child After = 500
Parent = 100
```